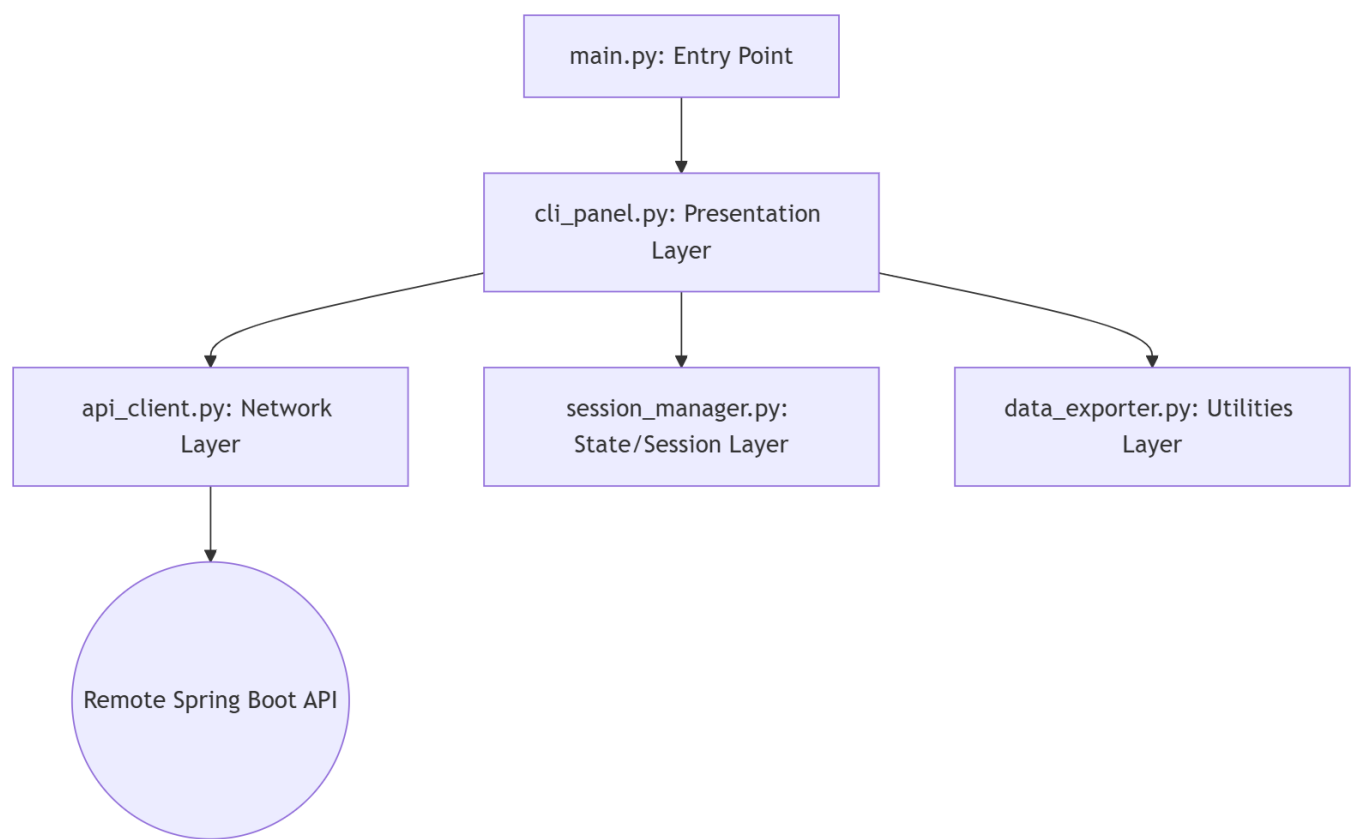


Student Grade Management CLI Portal - Architectural & System Guide

This document provides a detailed breakdown of the Python CLI client application designed to interface with the Pasadena City College (PCC) Student Grade Management backend. It serves as an architectural blueprint for understanding how the application operates, how security is enforced, and how to extend its features.

🔧 1. Architecture Overview

The client application follows a layered, modular architecture. This separation of concerns ensures that network request handling, local file writing, visual presentation, and state tracking are decoupled.



Component Directory

- **main.py:** Entrypoint script. Bootstrap for dependency injection and execution loop.
- **cli_panel.py:** Visual and interactive CLI layer. Renders dashboards, handles inputs, and runs validations.
- **api_client.py:** HTTP client abstraction. Manages JWT attachment and handles REST calls.
- **session_manager.py:** File-based caching layer for user credentials and logins.
- **data_exporter.py:** Ingestion/Export utility module. Handles CSV and JSON mapping.

🔒 2. Authentication & Session Flow

The CLI implements secure authentication using a **JSON Web Token (JWT)**. Sessions can be stored locally on the client's home directory.

Session Lifecycle

1. Auto-Login (Startup):

- The app invokes `SessionManager.load_session()` to read from `~/.pcc_session.json`.
- If a cached token exists, `CLIPanel` attempts to fetch the student directory.
- If successful (HTTP 200), the login screen is bypassed, routing the user straight to their dashboard.
- If the token is expired or the server returns `401 Unauthorized`, credentials are cleared and the user is redirected to the login screen.

2. Explicit Login:

- The user inputs their username/password.
- The user is prompted: *Enable 'Remember Me' for automatic login next time?*
- On a successful JWT response, the token, username, and role are cached (if "Remember Me" is enabled) or stored in memory.

3. Logout:

- The user has the option to clear saved credentials. Clearing them deletes `~/.pcc_session.json` and resets all memory variables.

3. Role-Based Access Control (RBAC)

The application handles three distinct user roles with strict boundary permissions. The frontend CLI coordinates menu choices and local access checks, but final security constraints are enforced by the Spring Boot backend server.

Feature / Action	Admin	Teacher	Student	Enforcement Layer
View Student Directory	Yes	Yes	Own Profile Only	API (JWT Scope check) & Local CLI Panel
Create Student Record	Yes	Yes	No	API & CLI (Menu option hidden)
Update Student Profile	Full	Partial (No Phone/Address)	No	API & CLI (Form prompts lock secure fields)
Delete Student Record	Yes	No	No	API & CLI (Menu option hidden)
View All Grades	Yes	Yes	No	API & CLI (Menu option hidden)
Modify Grade & Feedback	Yes	Yes	No	API & CLI (Menu option hidden)
Manage User Directory	Yes	No	No	API & CLI (Menu option hidden)
Bulk Import Grades (CSV)	Yes	Yes	No	API & CLI (Menu option hidden)

Frontend Field Protection (Example: Teacher Profile Update)

In `cli_panel.py`, when a `TEACHER` edits a student's profile, the CLI prompts for `firstName`, `lastName`, and `email`, but displays secure fields as `[LOCKED]`:

```
# Phone Number and Address are secure fields and locked for TEACHER role.  
self.console.print(f"Phone Number [LOCKED]: {student.get('phoneNumber')}")  
self.console.print(f"Home Address [LOCKED]: {student.get('address')}")
```

This UI restriction aligns with API server permissions to avoid a `403 Forbidden` error.

4. Testing & Quality Control

Verify correctness using `test_integration.py`.

The suite asserts:

1. **Authentication Success:** Simulates valid credentials to obtain a JWT.
2. **Authentication Failure:** Validates error messages on invalid logins.
3. **CRUD Integrity:** Inserts, updates, searches, and deletes student profiles.
4. **Role Access Control:** Tries to execute admin-only requests (e.g., creating user accounts) using a `STUDENT` or `TEACHER` session token and asserts that the API correctly rejects them with `403 Forbidden`.